National University of Computer & Emerging Sciences

Lecture 14

Scheduling II

Priority scheduling

- Not all jobs equal
  - So: rank them.
- Each process has a priority
  - Run highest priority ready job in system
- Priorities can be static or dynamic or both
- Among the Processes of equal priority
  - Round robin
  - FCFS

Priority scheduling

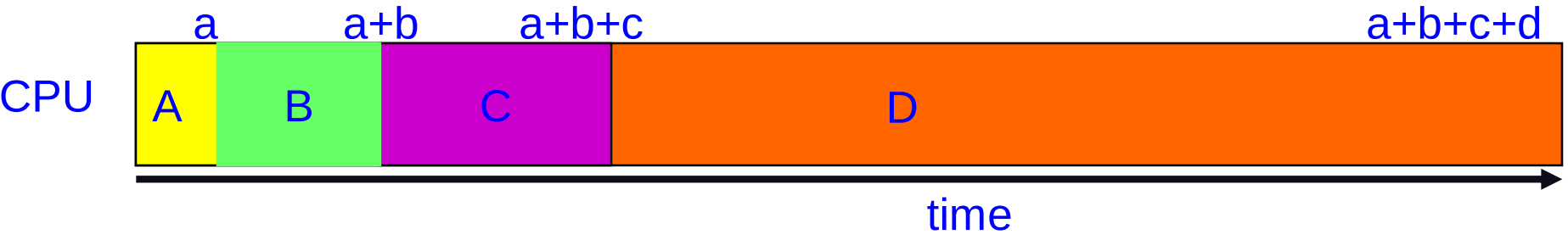
- Priority scheduling can be Preemptive or Non-Preemptive
- When a process arrives and enters the Ready Queue
- Its priority is compared with the currently Running Process
- If Higher
  - Preemptive Scheduling
    - Run the New process
  - Non-Preemptive Scheduling
    - Continue running the process

Priority scheduling

- High priority always runs over low priority.
- Starvation
  - A low Priority process may indefinitely wait for the CPU
- Solution: Aging
  - Gradually increase the Priority of processes that wait in the system for a long time.
- Which type of processes should be given Higher Priority
  - I/O Bound
  - CPU Bound
- In order to keep I/O busy increase priority for jobs that often block on I/O

Shortest Job First (SJF)

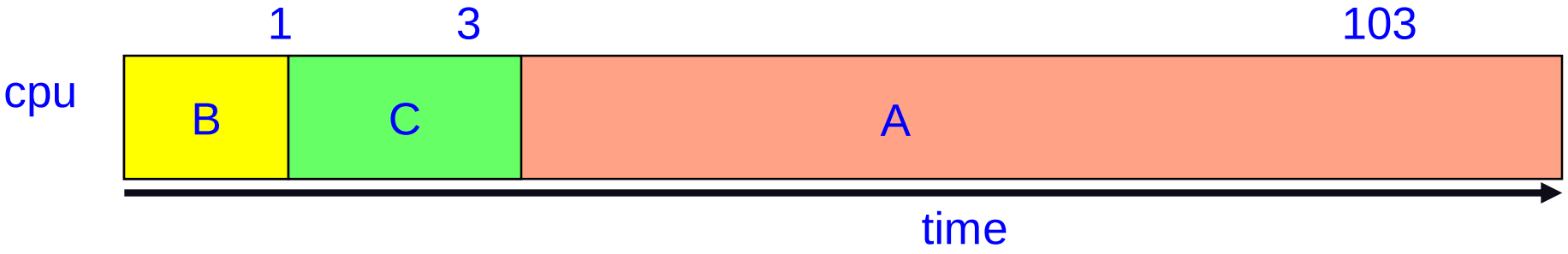
- Consider 4 jobs, a, b, c, d, run in lexical order



- The first (a) finishes at time a
- The second (b) finishes at time a+b
- The third (c) finishes at time a+b+c
- The fourth (d) finishes at time a+b+c+d
- Therefore average completion = (a + (a + b) + (a +b+c) + (a + b + c + d))/4 = (4a+3b+2c+d)/4
- Minimizing this requires a <= b <= c <= d.
- or Shortest Job First

Shortest Job First (SJF)

- Run whatever job has smallest next CPU burst
- Can be pre-emptive or non-pre-emptive
- Example: same jobs (given jobs A, B, C)

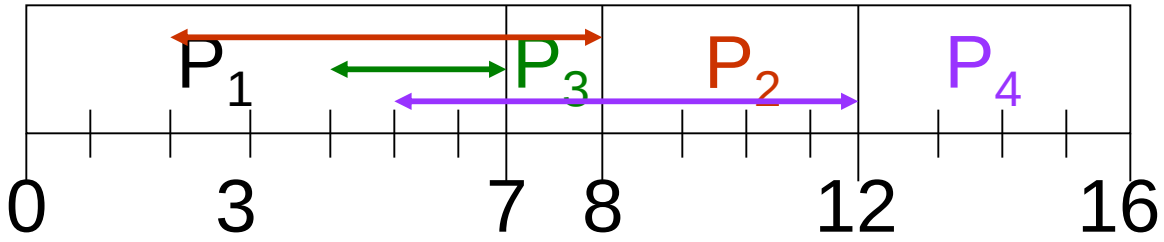


Average completion = (1+3+103) / 3 = ~35

Example of Non-Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)

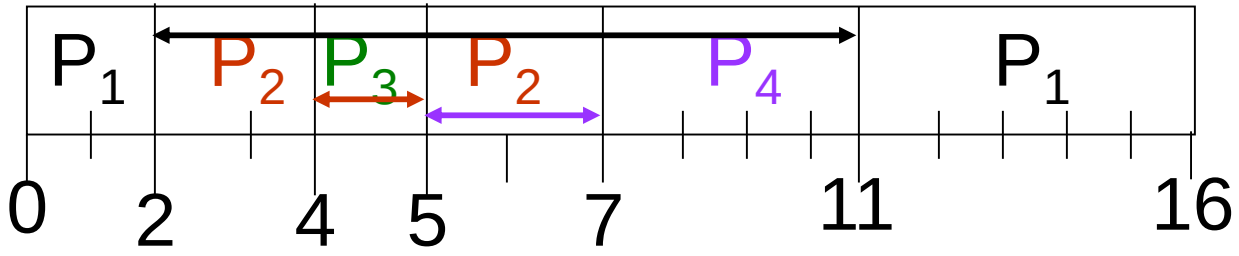


- Average waiting time
  - = (0 + 6 + 3 + 7)/4 = 4

Example of Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



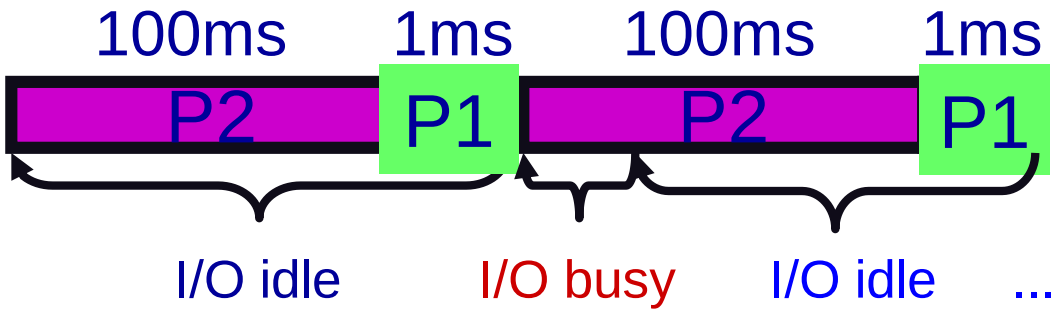
- Average waiting time
  - = (9 + 1 + 0 + 2)/4 = 3

SJF vs. RR

- Two processes P1, P2



- RR with 100ms time slice:



- I/O idle ~90%
- SJF Offers better I/O utilization

Shortest Job First

- The most important issue in SJF
  - Accuracy in estimation of Job length

Linux schedulers

- The 1.2 Linux scheduler used a circular queue for runnable task management that operated with a round-robin scheduling policy.

- Linux version 2.2 introduced the idea of scheduling classes, permitting scheduling policies for real-time tasks, non-preemptible tasks, and non-real-time tasks.

Linux schedulers

- The 2.4 kernel included a relatively simple scheduler that operated in O(N) time (as it iterated over every task during a scheduling event).

- The early 2.6 scheduler was called the *O(1) scheduler*

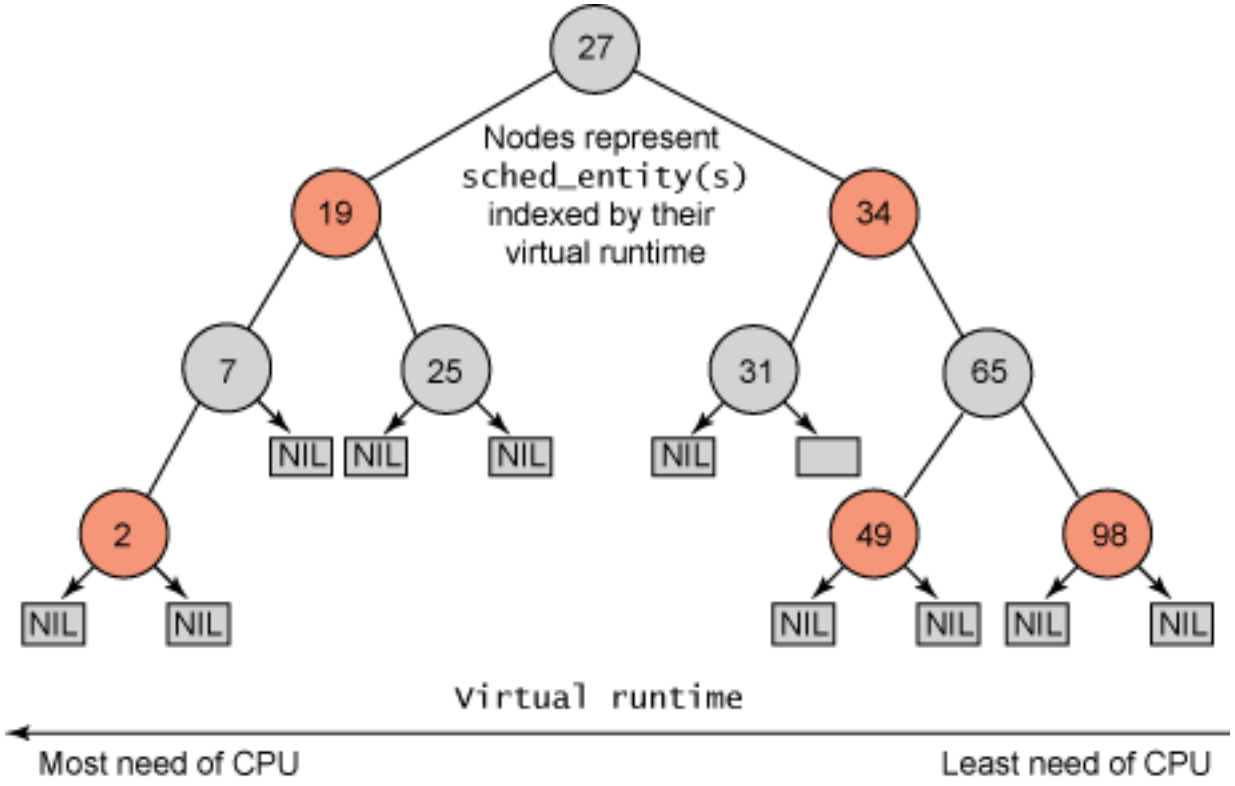
An overview of CFS

- The main idea behind the CFS is to maintain balance (fairness) in providing processor time to tasks.

- To determine the balance, the CFS maintains the amount of time provided to a given task in what's called the *virtual runtime.*
  - The smaller a task's virtual runtime the higher its need for the processor.

An overview of CFS

- Rather than maintaining the tasks in a run queue, as has been done in prior Linux schedulers, the CFS maintains a time-ordered red-black tree.


27
Nodes represent
sched_entity(s)
indexed by their
virtual runtime
19
34
7
25
31
65
NIL
NIL
NIL
NIL
NIL
2
49
98
NIL
NIL
NIL
NIL
NIL
NIL
Virtual runtime
Most need of CPU
Least need of CPU

An overview of CFS

- The scheduler then, to be fair, picks the left-most node of the red-black tree to schedule next to maintain fairness.

- The task accounts for its time with the CPU by adding its execution time to the virtual runtime and is then inserted back into the tree if runnable.

An overview of CFS

- In this way, tasks on the left side of the tree are given time to execute, and the contents of the tree migrate from the right to the left to maintain fairness.